

Vivlio — Documento de Handoff

Entrega da N1 • Setembro/2026 • branch develop

Este documento é o guia para o grupo que vai receber o Vivlio. Ele cobre como subir o sistema do zero, o que foi entregue, o que ficou de fora, onde estão as partes frágeis e onde encontrar cada coisa no código. O relatório de débitos técnicos é complementar a este documento.

1. Visão geral do sistema

O Vivlio é uma plataforma de troca de livros usados baseada em créditos: doar 1 livro gera 1 crédito, e 1 crédito permite resgatar 1 livro do acervo. A regra de negócio crítica é que ninguém resgata sem ter pelo menos 1 crédito ativo, e essa validação existe no backend (não só na tela).

Camada	Tecnologia
Backend	Java 17 + Spring Boot 3.3.5 (Web, Data JPA, Security, Validation) + JWT
Frontend	React 19 + Vite 8 + React Router 7 + Axios
Banco	PostgreSQL 16 (via Docker Compose); H2 em memória apenas nos testes
Repositório	Monorepo: pastas backend/ e frontend/; branch de integração é a develop

2. Como rodar o projeto do zero

2.1 Pré-requisitos

- JDK 17 ou superior (o projeto usa o Maven Wrapper, não é preciso instalar o Maven).
- Node.js 20.19+ ou 22.12+ (exigência do Vite 8) e npm.
- Docker e Docker Compose, para subir o PostgreSQL.

2.2 Banco de dados

Na pasta backend/, suba o PostgreSQL:

```
cd backend
docker compose up -d
```

O container sobe como vivlio-postgres na porta 5432, com banco, usuário e senha "vivlio", e healthcheck configurado. As tabelas são criadas automaticamente pelo Hibernate na primeira execução do backend — não há script SQL nem dados de exemplo.

2.3 Backend

```
cd backend
./mvnw spring-boot:run      # Linux/macOS
mvnw.cmd spring-boot:run    # Windows
```

A API sobe em <http://localhost:8080>. Para rodar os testes (usam H2, não precisam do Docker):

```
./mvnw test
```

2.4 Frontend

```
cd frontend
cp .env.example .env
```

```
npm install
npm run dev
```

A aplicação sobe em `http://localhost:5173` e consome a API pela variável `VITE_API_URL`.

2.5 Variáveis de ambiente

Todas têm valor padrão de desenvolvimento, então o sistema sobe sem configurar nada. Para outros ambientes, sobrescreva:

Variável	Padrão	Observação
DB_URL	jdbc:postgresql://localhost:5432/vivio	Conexão do backend
DB_USERNAME / DB_PASSWORD	vivio / vivlio	Iguais aos do docker-compose
JPA_DDL_AUTO	update	Ver ponto de atenção 5.1
JWT_SECRET	chave fixa de desenvolvimento	Trocar obrigatoriamente fora do local
JWT_EXPIRACAO_MS	86400000 (24h)	Validade do token
SERVER_PORT	8080	Porta da API
VITE_API_URL	http://localhost:8080	Frontend (arquivo .env)

Existe também o perfil `application-local.properties`, que só troca as credenciais do banco para `postgres/postgres` (útil para quem usa um PostgreSQL instalado na máquina, em vez do Docker). Ative com `--spring.profiles.active=local`.

2.6 Primeiro teste de ponta a ponta

O banco sobe vazio, então o acervo aparece sem livros e não é possível resgatar nada de cara. O caminho para validar a instalação é: cadastrar um usuário, doar um livro (ganha 1 crédito), cadastrar um segundo usuário, doar outro livro e então resgatar. A coleção `backend/postman/Vivio.postman_collection.json` cobre esses cenários pela API, incluindo os casos de erro.

3. O que está pronto (entregue na N1)

RF	Funcionalidade	Situação
RF01	Gestão de Contas	Cadastro, login com JWT e perfil (<code>/api/usuarios/me</code>). Telas de Login, Cadastro e Perfil. Senha com BCrypt.
RF02	Cadastro de Doação	POST <code>/api/livros</code> com título, autor e gênero validados. Tela de Doação.
RF03	Carteira de Créditos	+1 crédito na doação e -1 no resgate, com registro em <code>transacoes_credito</code> . Saldo exibido no Perfil.
RF04	Resgate de Livros	POST <code>/api/resgates</code> com a RN crítica validada no backend, transação atômica e lock pessimista contra concorrência.
RF05	Catálogo Público	GET <code>/api/livros</code> público, com busca por título/autor e filtro por gênero. Tela de Acervo.

Testes automatizados existentes: suíte JUnit no backend cobrindo autenticação, listagem, doação, resgate e um teste específico de resgate concorrente. Rodam com `./mvnw test`.

4. O que não está pronto

Item	Escopo	O que falta
RF06 — Avaliação de Conservação	N2	Nota inteira de 1 a 5 no cadastro do livro, exibida no card. Exige nova coluna estado_conservacao em livros, validação no backend (rejeitar 0 ou 6 com 400) e componente de estrelas no frontend.
RF07 — Clubes de Leitura	N3	Nada implementado. Exige novas tabelas de clubes e participantes, além de telas próprias.
Migrações de banco	—	Não há Flyway/Liquibase; o schema é gerado pelo Hibernate.
Pipeline de CI	—	Não existe .github/workflows; lint e testes rodam apenas localmente.
Testes de frontend	—	Nenhum framework de teste instalado no frontend.
Extrato de créditos	—	As transações são gravadas, mas não há endpoint nem tela para consultá-las.

5. Pontos de atenção

Nada no sistema é mock: o frontend conversa com a API real e a API grava no PostgreSQL. Os pontos abaixo são os lugares frágeis que merecem cuidado antes de mexer.

5.1 Schema gerado por ddl-auto=update

O Hibernate cria e altera as tabelas sozinho. Ele adiciona colunas novas, mas não remove nem altera as existentes, e não há histórico de versões. Como a avaliação de conservação exige uma coluna nova, o recomendado é adotar Flyway com um baseline do schema atual antes de qualquer mudança estrutural.

5.2 Segredo JWT com valor padrão no repositório

A chave de assinatura tem um fallback fixo em application.properties. Serve para desenvolvimento, mas qualquer ambiente exposto precisa definir JWT_SECRET. O token vale 24 horas, fica em localStorage e não há revogação: o logout só apaga o token no navegador.

5.3 Saldo de créditos existe em dois lugares

O valor corrente fica em usuarios.saldo_creditos e o histórico em transacoes_credito. Quem alterar saldo fora do CreditoService vai criar divergência silenciosa na regra crítica. Toda operação de crédito deve passar por CreditoService.creditar() ou debitar().

5.4 Concorrência no resgate depende de lock pessimista

ResgateService usa PESSIMISTIC_WRITE ao buscar usuário e livro (buscarParaAtualizarSaldo e buscarParaResgate). É o que impede saldo negativo e resgate duplicado, e é coberto pelo ResgateConcorrenteTest. Se refatorar o resgate, mantenha os locks e rode esse teste.

5.5 Gênero não é validado contra lista fixa

O frontend oferece um select com a lista de frontend/src/constants/generos.js, mas o backend aceita qualquer texto não vazio. Uma requisição direta à API pode gravar um gênero fora da lista e ele não aparecerá no filtro da tela.

5.6 Listagem do acervo sem paginação

GET /api/livros devolve todos os livros disponíveis de uma vez, com busca por LIKE sem índice. Funciona no volume atual, mas é o primeiro gargalo do RNF05 (resposta abaixo de 500ms) conforme o acervo crescer.

5.7 Usuário pode resgatar o próprio livro doado

Não há regra impedindo isso. A operação é neutra em créditos (+1 na doação, -1 no resgate), mas se isso for considerado um comportamento indevido, a validação precisa ser adicionada no ResgateService.

5.8 CORS restrito a localhost

A configuração em SecurityConfig libera apenas origens http://localhost:*. Ao publicar o frontend em outro domínio, é preciso incluir essa origem, senão as chamadas são bloqueadas pelo navegador.

6. Onde encontrar as coisas

O backend é organizado por domínio (auth, usuario, livro, credito, resgate, shared, config), cada um com entity, repository, service, controller e dto.

O que você procura	Onde está
Regra dos créditos (+1 / -1) e bloqueio sem saldo	backend/.../credito/service/CreditoService.java
Fluxo do resgate e RN crítica	backend/.../resgate/service/ResgateService.java
Fluxo da doação	backend/.../livro/service/LivroService.java
Login, registro e hash de senha	backend/.../auth/service/AuthService.java
Geração e leitura do token JWT	backend/.../auth/service/JwtService.java
Quem lê o token a cada requisição	backend/.../auth/security/JwtAuthenticationFilter.java
Rotas públicas x protegidas e CORS	backend/.../config/SecurityConfig.java
Tratamento de erros da API	backend/.../shared/exception/ApiExceptionHandler.java
Consultas e locks de banco	backend/.../livro/repository/LivroRepository.java e usuario/repository/UsuarioRepository.java
Configuração e variáveis	backend/src/main/resources/application.properties
Testes automatizados	backend/src/test/java/br/com/vivlio/
Rotas do frontend	frontend/src/App.jsx
Sessão, login e saldo em memória	frontend/src/context/AuthProvider.jsx
Cliente HTTP, token e tratamento de 401	frontend/src/services/api.js
Telas	frontend/src/pages/ (Login, Cadastro, Acervo, Doacao, Perfil, NaoEncontrada)
Proteção de rota	frontend/src/components/RotaProtegida.jsx

6.1 Endpoints da API

Método e rota	Acesso	Função
POST /api/auth/regar	Público	Cria conta (saldo inicia em 0) e devolve token

Método e rota	Acesso	Função
POST /api/auth/login	Público	Autentica e devolve token
GET /api/usuarios/me	Token	Perfil e saldo do usuário logado
GET /api/livros	Público	Lista o acervo disponível (params: busca, genero)
POST /api/livros	Token	Doa um livro e credita +1
POST /api/resgates	Token	Resgata um livro e debita 1 crédito

7. Sugestão de por onde começar

- 1 Subir o ambiente pelo passo 2 e validar o fluxo doar → resgatar usando a coleção do Postman.
- 2 Rodar ./mvnw test para confirmar que a suíte passa na máquina de vocês.
- 3 Ler CreditoService e ResgateService — é onde está a regra que não pode quebrar.
- 4 Adotar Flyway com baseline do schema atual antes de criar a coluna de avaliação do RF06.
- 5 Só então implementar o RF06, aproveitando que o formulário de doação e o card do acervo já existem.

Documentos complementares

- README.md — visão geral, escopo e equipe.
- CONTRIBUTING.md — fluxo de branches, padrão de commits e regras de Pull Request.
- Docs/Documento de Especificação de Requisitos e DoR.docx — RFs, RNFs, histórias e critérios de aceitação.
- Docs/Plano_de_Testes_Sebo_Comunitario.pdf — casos de teste (TC01 a TC13).
- Docs/Vivlio - Debitos Tecnicos.docx — os cinco débitos técnicos priorizados.
- backend/postman/Vivlio.postman_collection.json — coleção pronta de requisições da API.